

```
rm(list = ls(all = TRUE))
#Do not delete this!
#It clears all variables to ensure reproducibility
```

South American frogs in JAGS

We will be using a subset of data from the Anuran Calls repository. The data consists of 6882 individual syllables found in the recordings of frog calls; there are 55 unique frogs in this dataset, and their ID is provided as `recordID`. For each syllable, the audio signal was decomposed into 22 Mel-frequency cepstrum coefficients (MFCCs) (already removed the first coefficient). Each coefficient describes the spectral decomposition of the audio signal, i.e., its shape. The dataset also includes the frogs' family, genus, and species.

```
#Load in the data
frogs<-read.csv("Frogs.csv")
```

We are going to perform a Bayesian clustering of the MFCC values in order to identify different frogs.

JAGS model to fit a bivariate normal distribution, with the following prior specification:

let $\mathbf{Y}_i = (Y_{1i}, Y_{2i}) \sim \text{BVN}(\boldsymbol{\mu}, \Sigma)$, where

$$\begin{aligned}\boldsymbol{\mu} &\sim \text{BVN}(\mathbf{0}, 100^2 * \mathbf{I}_2), \\ \Sigma_{11} &= \sigma_1^2, \quad \Sigma_{22} = \sigma_2^2, \quad \Sigma_{12} = \Sigma_{21} = \sigma_1 \sigma_2 \rho, \\ \rho &\sim \text{Unif}(-1, 1), \quad \sigma_1^2, \sigma_2^2 \sim \text{Inverse-Gamma}(1, 0.1).\end{aligned}$$

We will apply it to the `MFCCs_2` and `MFCCs_11` coefficients for frogs 1 and 46.

```
library(rjags)

## Loading required package: coda

## Linked to JAGS 4.3.1

## Loaded modules: basemod,bugs

index_1_46 <- which(frogs$RecordID == 1 | frogs$RecordID == 46 )
Y <- cbind(frogs[index_1_46,]$MFCCs_2, frogs[index_1_46,]$MFCCs_11)

model_string_1 <- "model{

  tau1 ~ dgamma(a,b)
  tau2 ~ dgamma(a,b)

  rho ~ dunif(c,d)

  mu ~ dmnorm( mu0, tau0 )

  prec[1,1] <- tau1 / (1-rho^2)
  prec[2,2] <- tau2 / (1-rho^2)
  prec[1,2] <- -rho * sqrt(tau1 * tau2) / (1-rho^2)
  prec[2,1] <- prec[1,2]
```

```

for(i in 1:n){
  Y[i,] ~ dmnorm( mu, prec )
}

sigma_sq1 <- 1/tau1
sigma_sq2 <- 1/tau2
}"

mu0 <- c(0,0)
n <- nrow(Y)
tau0 <- diag(0.0001,2)
a <- 1
b <- 0.1
c <- -1
d <- 1

data <- list(Y = Y, mu0 = mu0, tau0 = tau0, a = a, b = b, c = c, d = d, n=n )

model1 <- jags.model(textConnection(model_string_1), n.chains = 1, data = data)

```

```

## Compiling model graph
##   Resolving undeclared variables
##   Allocating nodes
## Graph information:
##   Observed stochastic nodes: 73
##   Unobserved stochastic nodes: 4
##   Total graph size: 104
##
## Initializing model

```

```

update(model1, 4000, progress.bar = "none")

res1 <- coda.samples(model1,
  variable.names = c("mu","sigma_sq1","sigma_sq2","rho"),
  n.iter = 10000,
  thin = 5, progress.bar="none")

summary(res1)

```

```

##
## Iterations = 5005:15000
## Thinning interval = 5
## Number of chains = 1
## Sample size per chain = 2000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##           Mean          SD Naive SE Time-series SE
## mu[1]      0.25843 0.017636 3.943e-04      3.943e-04
## mu[2]      0.16513 0.020344 4.549e-04      4.719e-04
## rho       -0.79982 0.043533 9.734e-04      1.117e-03
## sigma_sq1  0.02263 0.003810 8.518e-05      9.249e-05

```

```
## sigma_sq2  0.03072 0.005135 1.148e-04      1.303e-04
##
## 2. Quantiles for each variable:
##
##           2.5%      25%      50%      75%      97.5%
## mu[1]      0.22489  0.24650  0.25811  0.27009  0.29229
## mu[2]      0.12526  0.15152  0.16558  0.17896  0.20401
## rho       -0.87354 -0.83110 -0.80492 -0.77336 -0.70173
## sigma_sq1  0.01652  0.01996  0.02216  0.02507  0.03106
## sigma_sq2  0.02217  0.02720  0.03009  0.03372  0.04251
```

A Bayesian analysis using a bivariate normal model has been used to examine the relationship between the coefficients MFCCs_2 and MFCCs_11 extracted from frog 1 and 46. The model is built using JAGS (that evaluates the posterior densities using MCMC methods).

In order to give to σ_1^2 and σ_2^2 a *Inverse – Gamma*(1,0.1) prior, a prior *Gamma*(1,0.1) has been assigned to τ_1 and τ_2 (the respective precisions).

Since we are working with *dmnorm()* to assign a multivariate normal prior to both $\mathbf{Y}$ and $\boldsymbol{\mu}$, the precision matrix has to be given as argument to the function: for $\boldsymbol{\mu}$, since μ_1 and μ_2 are independent, the diagonal matrix with $1/100^2$ as diagonal element has been used, on the other hand for $\mathbf{Y}$ the inverse of the covariance matrix has been computed in the model.

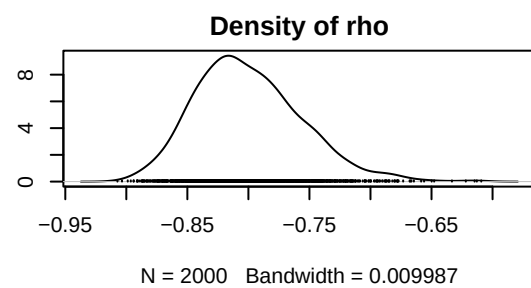
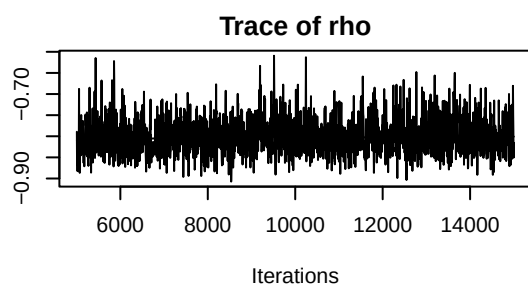
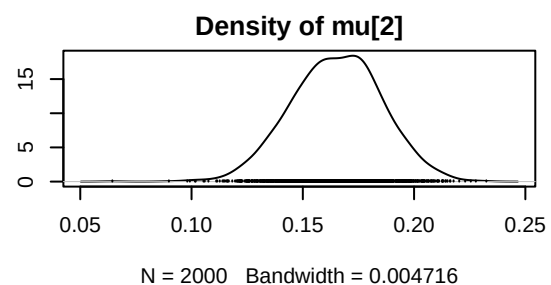
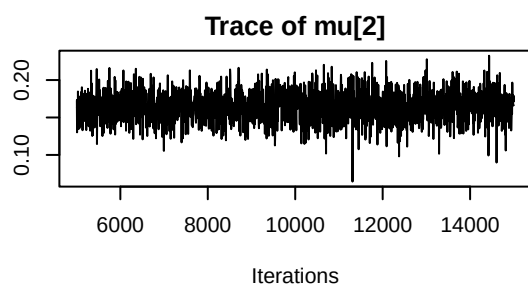
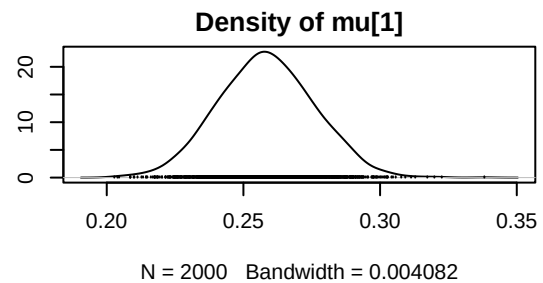
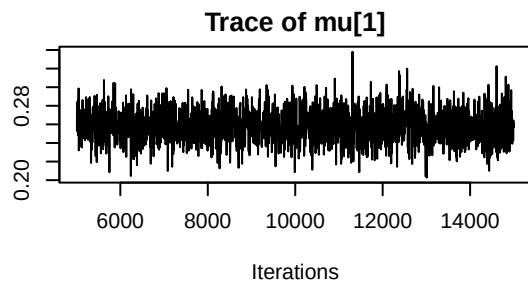
The summary indicates how the coefficients MFCCs_2 and MFCCs_11 are centered around 0.25 and 0.17, with the estimate for μ_2 that shows more variability than the one for μ_1 (higher standard deviation, meaning wider CI for the mean).

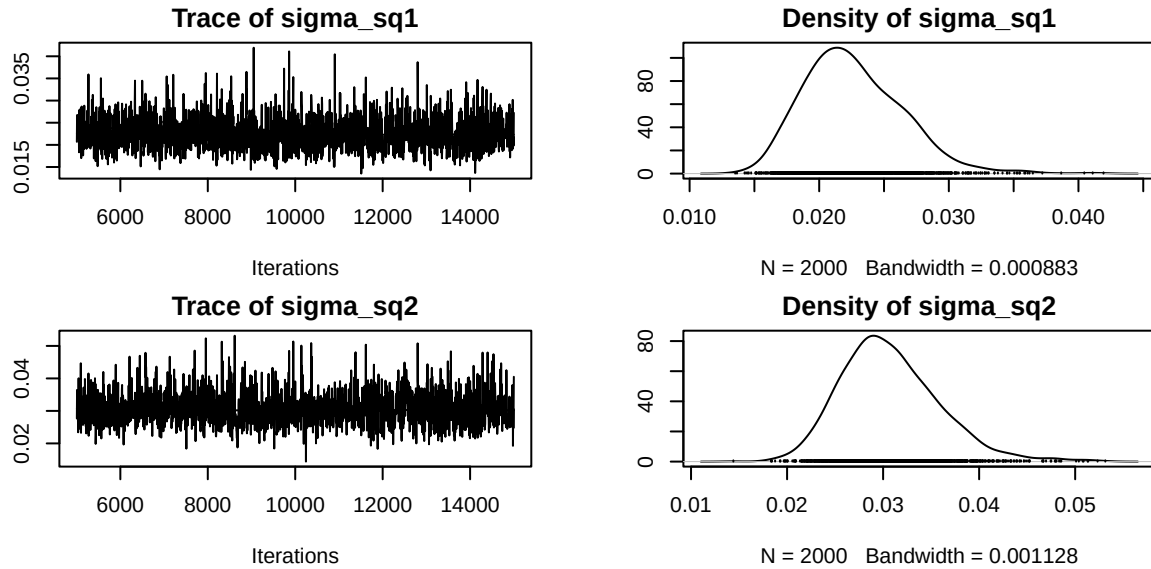
Also, the variance estimate show that the mean value for σ_1^2 is lower than the one for σ_2^2 , meaning that the variability in the first component (MFCCs_2) is lower than in the second(MFCCs_11), but since the CI at 95% confidence of the two estimates overlap, this conclusion cannot be confirmed.

The results confirm a strong negative correlation between the two features (ρ is negative and its mean is close to -0.9), indicating an inverse relationship in their variations. Also, the Bayesian CIs for ρ are strictly negative as well, and quite narrow, always indicating a negative correlation of at least -0.7.

In order to check for mixing and convergence, trace plots and posterior densities can be visualized:

```
par(mfrow = c(3,2), mar = c(4, 4, 2, 2))
plot(res1)
```





```
par(mfrow = c(1,1))
```

The trace and density plots show good mixing and stationarity, suggesting that the chains have converged, especially for μ_1 and μ_2 . The mixing for ρ seems slightly slower than the other parameters.

The posterior density distributions are smooth and unimodal, meaning that the parameter estimation should be reliable.

The strong negative correlation between MFCCs_2 and MFCCs_11 is confirmed by the posterior density of ρ , that has domain on strictly negative values.

Overall, the MCMC process seems to have been efficient.

Want the effective sample size for ρ above 2500.

```
#Model with 3 chains
model2 <- jags.model(textConnection(model_string_1), n.chains = 3, data = data)
```

```
## Compiling model graph
##   Resolving undeclared variables
##   Allocating nodes
## Graph information:
##   Observed stochastic nodes: 73
##   Unobserved stochastic nodes: 4
##   Total graph size: 104
##
## Initializing model
```

```

#Increase burn-in
update(model1, 4000, progress.bar = "none")
#Increasing number of iteration
res2 <- coda.samples(model2,
                      variable.names = c("mu","rho"),
                      n.iter = 6000, progress.bar="none")
#computing effective size
ess_val<-effectiveSize(res2)

cat("Effective size for rho is:", ess_val["rho"])

```

```
## Effective size for rho is: 3831.231
```

In order to obtain the desired effective size for ρ , the JAGS model is fitted again, this time increasing the number of chains to 3.

From the previous residual plot the burn-in seemed to allow convergence of the chains, so it has not been modified (4000).

The increased number of chains allow the model to have more samples, meaning that more independent information is at hand. So compared to the previous model the number of samples has been reduced to 6000 for each chain.

This setting allows to obtain an effective size for ρ higher than 2500.

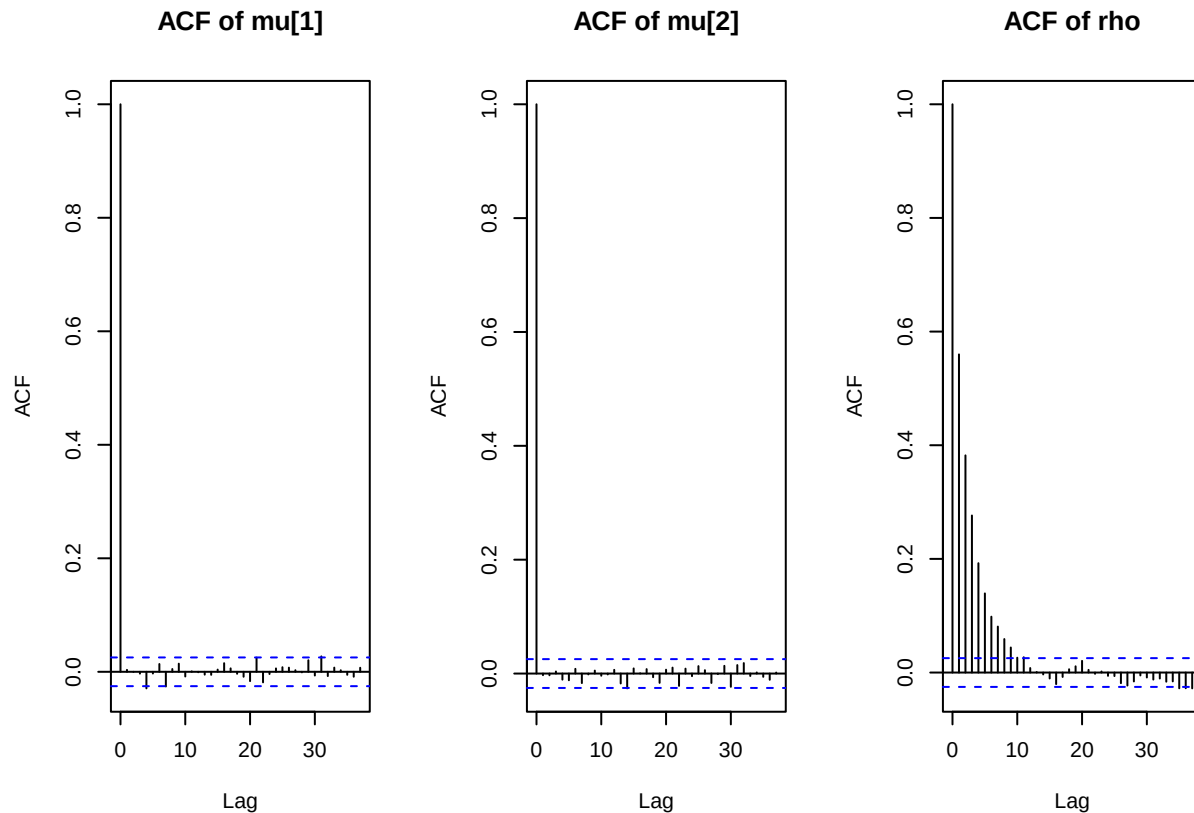
Note: thinning, present at the previous step, has been removed.

Let's now have a look to autocorrelation:

```

par(mfrow = c(1,3))
acf(res2[[1]][,1], main = "ACF of mu[1]")
acf(res2[[1]][,2], main = "ACF of mu[2]")
acf(res2[[1]][,"rho"], main = "ACF of rho")

```



The ACF plots show that the autocorrelation for some parameters (μ_1 and μ_2) drops quickly, indicating that the MCMC chains are mixing well and producing independent samples. On the other hand, for ρ the autocorrelation drops slower.

Note: autocorrelation at lag 0 is always 1 (for definition of ACF).

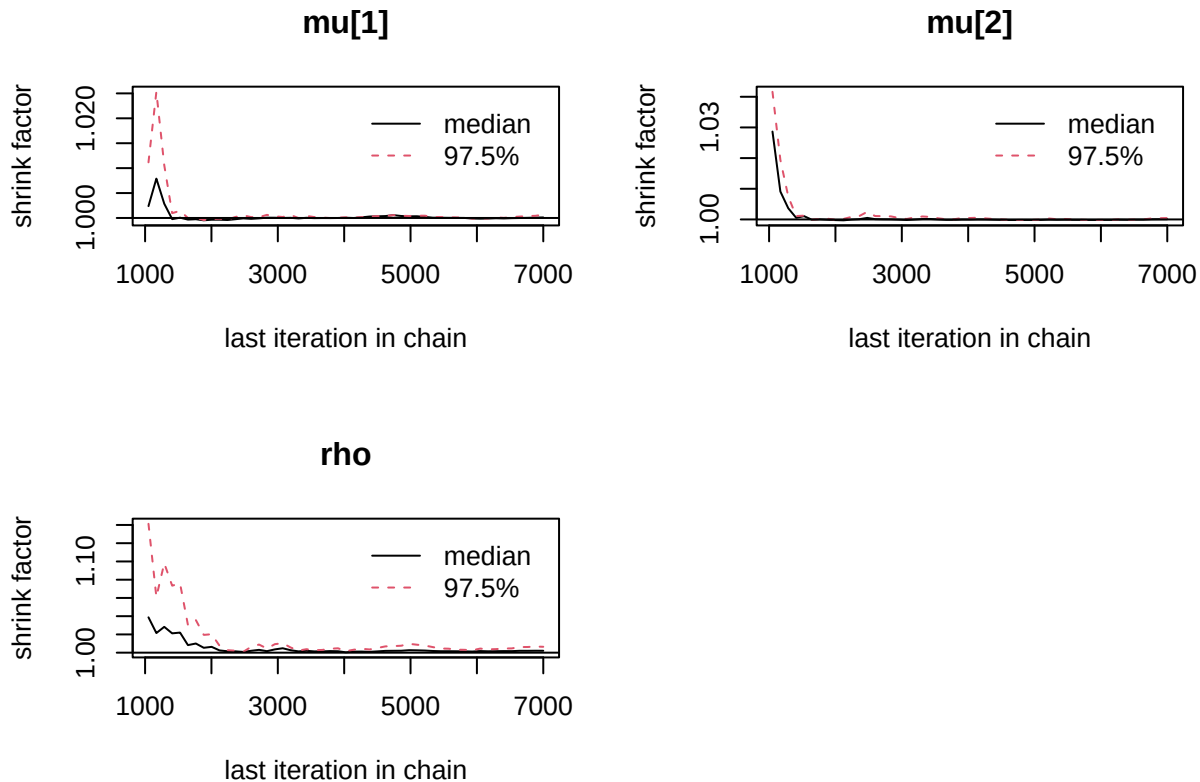
After lag 1, for μ_1 and μ_2 the autocorrelation goes quickly to very low values (around 0).

For ρ there is a small amount of autocorrelation at low lags as well, that slowly decreases towards 0 before lag 15. This means that the samples for ρ are dependent of the previous states, leading to the slow mixing that was observed in the trace plot above. ρ might require some thinning in order to get more independent samples.

Despite this problem, the ACF plot seems to stabilize in a reasonable number of lags, meaning that the model can be considered reliable.

To go on with diagnostic, it's possible to evaluate Gelman-Rubin plots and statistics:

```
gelman.plot(res2)
```



The Gelman-Rubin diagnostic plots indicate that the MCMC chains have converged well, as the shrink factor ($\hat{R}$) stabilizes near 1 for all the parameters.

Remember, this does not prove convergence, but proves that all the chains are behaving in a similar way (same within standard deviation in the different chains). Then if $\hat{R}$ is around 1, it can be interpreted as a proof that the chains have not diverged.

For all the parameters, initially the value of $\hat{R}$ is higher, stressing how the different chains behave differently at the start of the sampling (different initialisation).

After convergence of $\hat{R}$, for all parameters, both the median and the 97.5% quantile lines remain close to 1, confirming that no significant between-chain variation remains. For ρ and μ_2 the 97.5% quantile line shows some fluctuations, but nothing major.

Overall, these results provide strong evidence that the model has reached stationarity and that the posterior estimates can be considered reliable.

```
gelman.diag(res2)
```

```
## Potential scale reduction factors:
##
##      Point est. Upper C.I.
## mu[1]          1      1.00
## mu[2]          1      1.00
## rho            1      1.01
##
## Multivariate psrf
##
```


1

$\hat{R}$ for all parameters is exactly 1 (as the upper confidence intervals), confirming that the MCMC chains have reached stationarity and the results can be considered reliable.

We are now going to fit a bivariate normal mixture model. For K mixture components, our model will have the form:

$$\begin{aligned} \mathbf{Y}_i | (Z_i = k) &\sim BVN(\boldsymbol{\mu}^{(k)}, \Sigma) \\ \Pr\{Z_i = k\} &= p_{i,k}, \quad \mu_1^{(k)} \sim N(0, 100^2), \quad \mu_2^{(k)} \sim N(0, 100^2), \quad k = 1, \dots, K, \\ \sum_{k=1}^K p_{i,k} &= 1, \quad \text{for all } i = 1, \dots, N, \\ \Sigma_{11} &= \sigma_1^2, \quad \Sigma_{22} = \sigma_2^2, \quad \Sigma_{12} = \Sigma_{21} = \sigma_1 \sigma_2 \rho, \\ \rho &\sim \text{Unif}(-1, 1), \quad \sigma_1^2, \sigma_2^2 \sim \text{Inverse-Gamma}(1, 0.1), \end{aligned}$$

where all $\mu_j^{(k)}, j = 1, 2, k = 1, \dots, K$ are independent a priori.

The latent cluster label Z_i tells us which cluster observation $\mathbf{Y}_i$ belongs to. Note that the mean vector of the bivariate normal distribution changes with the cluster, but the covariance matrix is the same for all clusters.

Write a JAGS model for a bivariate normal mixture with $K = 2$ components (basically same as before). Note that your model will suffer from label switching if you do not impose any constraints on the parameters (see this article). This can easily be circumvented by ordering the first component of the mean vector, i.e., by forcing $\mu_1^{(1)} \leq \mu_1^{(2)} \leq \dots \leq \mu_1^{(K)}$.

Fit this model to the same data as before. We will use a Beta(0.5,0.5) prior on $p_{1,i}, i = 1, \dots, N$; note that $p_{i,2} = 1 - p_{i,1}$.

```
model_string_2k <- "model{

  #firs element of mu, sampled k times and then sorted

  for(i in 1:2){
    mu1_start[i] ~ dnorm(mu0, tau0)
    mu2[i] ~ dnorm(mu0, tau0)
  }

  mu1[1:2] <- sort(mu1_start[1:2])
  #precision for sigma squared 1 and 2
  tau1 ~ dgamma(a,b)
  tau2 ~ dgamma(a,b)

  #rho
  rho ~ dunif(c,d)

  #compute precision matrix (inverse of covariance matrix)
  prec[1,1] <- tau1 / (1-rho^2)
  prec[2,2] <- tau2 / (1-rho^2)
  prec[1,2] <- -rho * sqrt(tau1 * tau2) / (1-rho^2)
  prec[2,1] <- prec[1,2]

  #Matrix of mu used later (every row is a vector for mu, with K=2 rows)
```

```

for (k in 1:2) {
  mu[k,1] <- mu1[k] # First dimension mean
  mu[k,2] <- mu2[k] # Second dimension mean
}

for(i in 1:n){
  #each observation has a different probability for clusters:
  p[i,1] ~ dbeta( e, f)
  p[i,2] <- 1 - p[i,1]

  #cluster for observation i
  Z[i] ~ dcat(p[i,1:2])

  Y[i,1:2] ~ dmnorm( mu[Z[i],1:2], prec )
}

sigma_1 <- sqrt(1/tau1)
sigma_2 <- sqrt(1/tau2)
}"

mu0 <- 0
n <- nrow(Y)
tau0 <- 1/(100^2)
a <- 1
b <- 0.1
c <- -1
d <- 1
e <- 0.5
f <- 0.5

Y_scaled <- scale(Y)

data2 <- list(Y = Y_scaled, mu0 = mu0, tau0 = tau0, a = a, b = b, c = c,
             d = d, n=n, e=e, f = f)

model_k2 <- jags.model(textConnection(model_string_2k),
                      n.chains = 1, data = data2)

## Compiling model graph
##   Resolving undeclared variables
##   Allocating nodes
## Graph information:
##   Observed stochastic nodes: 73
##   Unobserved stochastic nodes: 153
##   Total graph size: 476
##
## Initializing model

update(model_k2, 5000, progress.bar = "none")

```

The JAGS model “model_string_k_2” implements a bivariate normal mixture model with $K=2$, meaning that each observation Y is assigned to one of two clusters, based on latent categorical variables Z .

Each cluster, has a different mean μ (bidimensional vector). The first elements of the mean vector for each cluster ($\mu_1^{(k)}$) are sampled from a normal prior (using `dnorm`) with mean 0 and precision $1/100^2$, since they are independent and equally distributed. Then, to prevent label identifiability, the vector of the μ_1 is sorted.

The second elements of the mean vector ($\mu_2^{(k)}$) are sample from normal prior as well, with the same parameters as $\mu_1^{(k)}$.

σ_1^2 , σ_2^2 , and ρ follow the same priors as the previous step (problem 2.1).

Each observation Y_i (bidimensional) is assigned with a probability to be a member of each cluster, in our case $K=2$ and we got: $p[i,1] \sim \text{beta}(0.5, 0.5)$ and $p[i,2] = 1 - p[i,1]$ (to ensure that the sum of the probabilities is 1). For each observation, a latent variable is sampled using a categorical distribution, based on the probabilities $p[i]$.

Each Y_i follows a bivariate normal distribution with the assigned cluster mean and the (the same for every observation i) precision matrix.

The model aims to estimate cluster means for each observation, along with variances and correlation for the elements of $Y_i = (Y_{1,i}, Y_{2,i})$.

```
res2k <- coda.samples(model_k2,
  variable.names = c("sigma_1", "sigma_2", "rho", "Z[1]",
    "Z[48]", "p[1,1]", "p[48,2]", "mu"),
  n.iter = 2500*5,
  thin = 5, progress.bar="none")
summary(res2k)
```

```
##
## Iterations = 6005:18500
## Thinning interval = 5
## Number of chains = 1
## Sample size per chain = 2500
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##           Mean      SD Naive SE Time-series SE
## Z[1]      1.0000 0.00000 0.0000000      0.000000
## Z[48]     1.6332 0.48203 0.0096406      0.036882
## mu[1,1]  -0.5427 0.09538 0.0019077      0.003656
## mu[2,1]   1.1779 0.13476 0.0026951      0.003891
## mu[1,2]   0.6334 0.04960 0.0009920      0.001231
## mu[2,2]  -1.3828 0.08598 0.0017197      0.003596
## p[1,1]    0.7586 0.24379 0.0048759      0.005876
## p[48,2]   0.5758 0.34545 0.0069089      0.021253
## rho      -0.1646 0.19544 0.0039088      0.013635
## sigma_1   0.6016 0.07697 0.0015394      0.004984
## sigma_2   0.3401 0.03690 0.0007381      0.002717
##
## 2. Quantiles for each variable:
##
##           2.5%      25%      50%      75%      97.5%
## Z[1]      1.000000  1.0000  1.0000  1.000000  1.0000
```

```
## Z[48]      1.000000  1.0000  2.0000  2.000000  2.0000
## mu[1,1] -0.718391 -0.6102 -0.5448 -0.479256 -0.3508
## mu[2,1]  0.902022  1.0921  1.1796  1.268941  1.4335
## mu[1,2]  0.540133  0.5998  0.6331  0.665295  0.7308
## mu[2,2] -1.538952 -1.4432 -1.3853 -1.327405 -1.2071
## p[1,1]   0.162177  0.6214  0.8434  0.963933  0.9997
## p[48,2]  0.003261  0.2508  0.6477  0.908368  0.9991
## rho      -0.519516 -0.3252 -0.1523 -0.006433  0.1728
## sigma_1   0.473413  0.5411  0.5957  0.654507  0.7627
## sigma_2   0.282297  0.3131  0.3358  0.361269  0.4249
```

The bivariate normal mixture with two clusters is implemented, with a number of iterations equals to 2500×5 , in order to get 2500 samples, since the thinning is set to 5.

The model assigns each observation Y_i to a cluster, indicated by the latent variable Z_i .

The summary shows that cluster 1 has mean around $(-0.55, 0.64)$ and cluster 2 has a mean around $(1.18, -1.38)$, with CI that for the components of the means do not overlap, suggesting a good separation of the two clusters.

The standard deviations, refer to the variability in the elements of Y , suggest a wider spread in the first component (MFCCs_2), since $\sigma_1 > \sigma_2$ (even here CIs do not overlap, so this conclusion is statistically strong).

The correlation parameter ρ is estimated around -0.14, implying weak negative relationship between the two coefficients MFCCs_2 and MFCCs_11 (elements of Y).

Compared to the previous model the interpretation of the parameters has strongly changed: a mean for each cluster is now present $\mu^{(1)}$ and $\mu^{(2)}$. Even if two clusters are now present, the observations Y_i share the same covariance matrix, but now this does not capture the overall variances and correlation of the model, but it's a measure of variability and correlation among MFCCs_2 and MFCCs_11 within the clusters (assumed to be the same in all the clusters). That's why the values of ρ and the variances are changed so much, compared to the previous model.

In this sampling, the focus is on the cluster assignment of observation Y_1 and Y_{48} . Y_1 is consistently assigned to cluster 1, with mean value of Z_1 equal to 1 and all CIs equals to 1 as well, meaning that there is no variability across the assignment to the cluster of Y_1 . On the other hand, Y_{48} shows more uncertainty, since Z_{48} shows a mean value around 1.6, and CIs that span between 1 and 2; meaning that Y_{48} switches clusters between the different iterations.

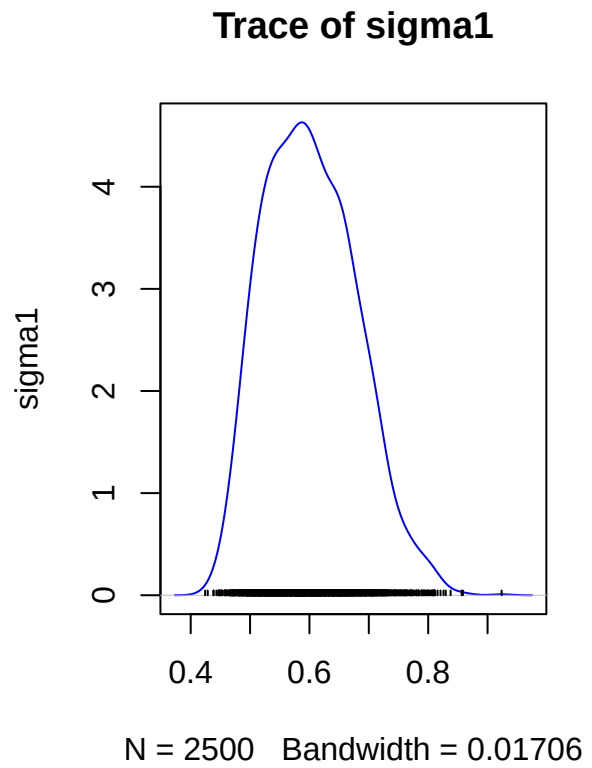
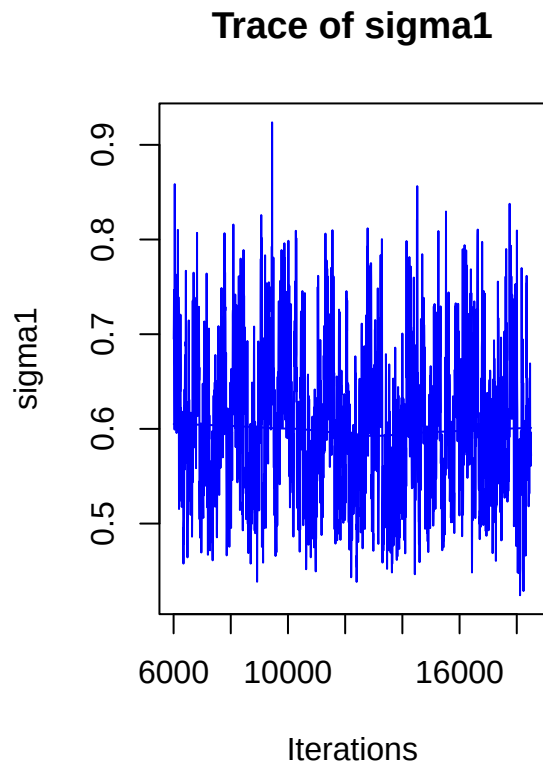
The probability estimates confirm this: $p_{(1,1)}$ shows a mean value of 0.75 with 50% quantile above 0.8, meaning that Y_1 has a high probability of belonging to cluster 1. On the other hand, Y_{48} shows a probability $p_{(48,2)}$ of mean value around 0.59, meaning that the assignment of this observation to the cluster is more uncertain.

All this suggest that some observations are properly captured by one of the two clusters, while others are difficult to classify. This could be a hint of more clusters needed to properly capture the structure of the data.

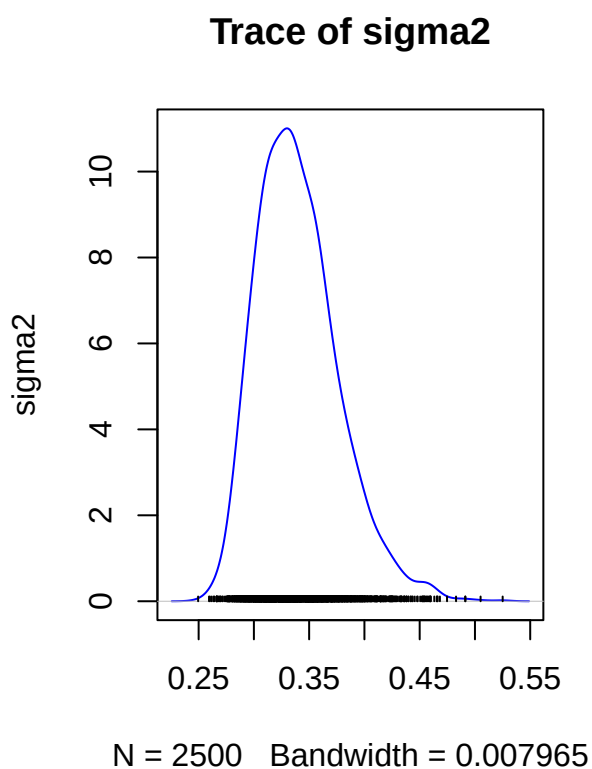
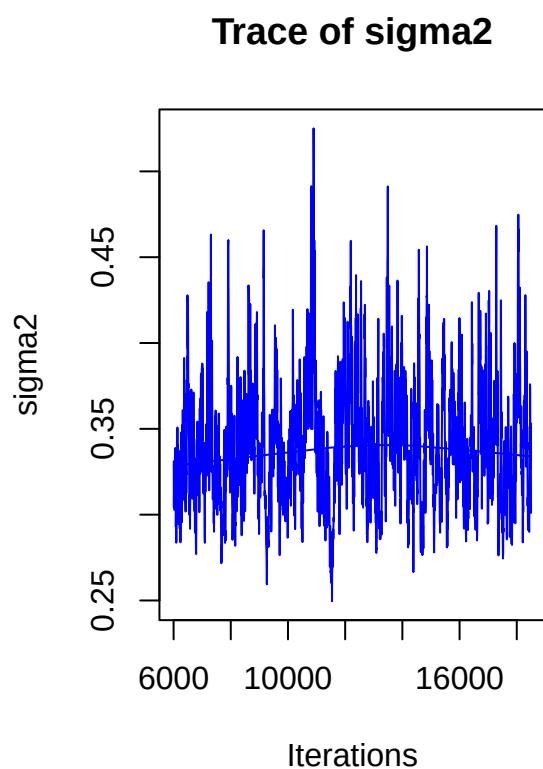
Now, the focus moves on the trace plots and posterior densities of σ_1 , σ_2 and ρ :

```
par(mfrow = c(2,3))

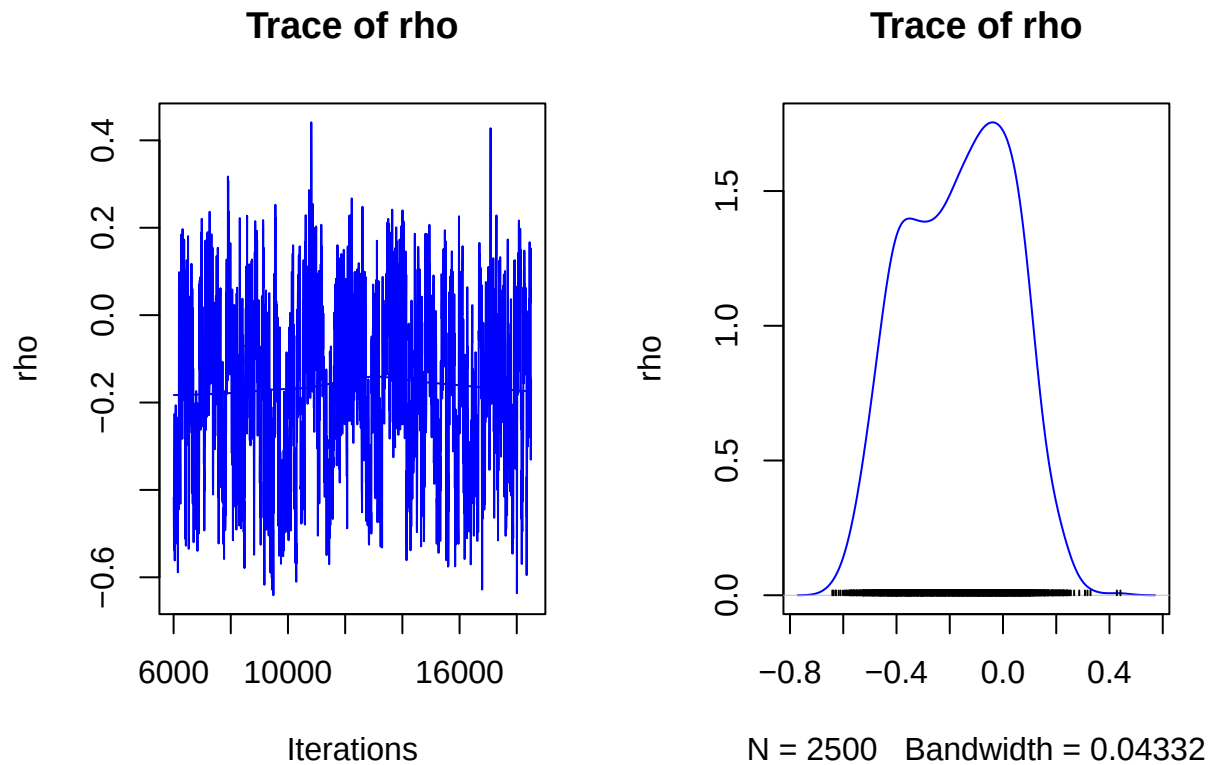
# Trace plots for sigma1, sigma2, and rho
plot(res2k[, "sigma_1"], main = "Trace of sigma1",
     ylab = "sigma1", col = "blue")
```



```
plot(res2k[, "sigma_2"], main = "Trace of sigma2",  
     ylab = "sigma2", col = "blue")
```



```
plot(res2k[, "rho"], main = "Trace of rho",  
     ylab = "rho", col = "blue")
```



```
par(mfrow = c(1,1))
```

The trace plots for σ_1 and σ_2 show good mixing, suggesting that the MCMC have explored the parameter space in a good way.

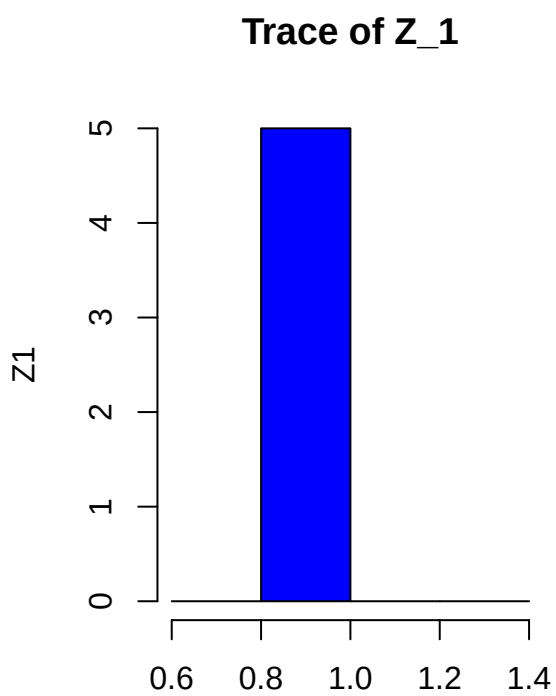
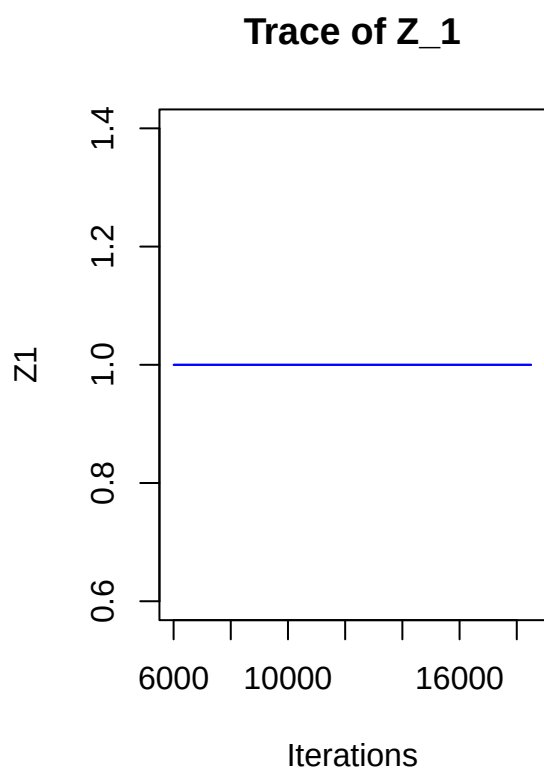
The posterior densities for both the standard deviations are unimodal, with σ_1 centered around 0.6 and σ_2 around 0.35, indicating, as previously stated, different level of variability around MFCCs_2 (higher) and MFCCs_11 (lower).

On the other hand ρ has a more problematic posterior distribution, with two local maximums around -0.4 and 0, giving uncertainty in the relationship between the two coefficients. The trace plot shows good mixing.

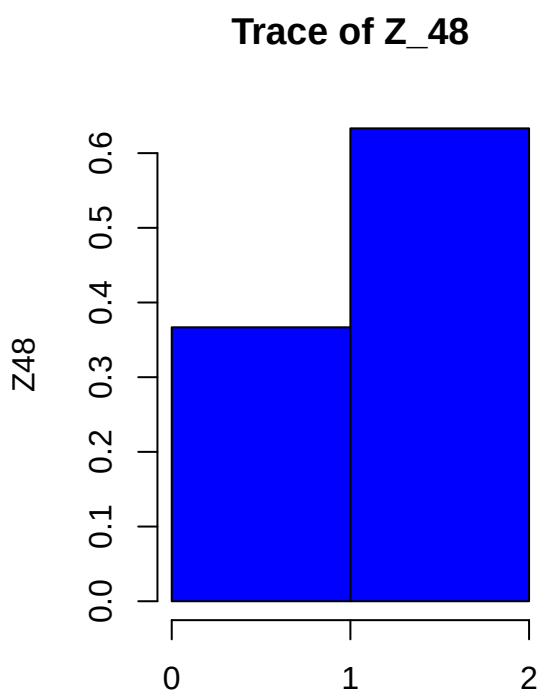
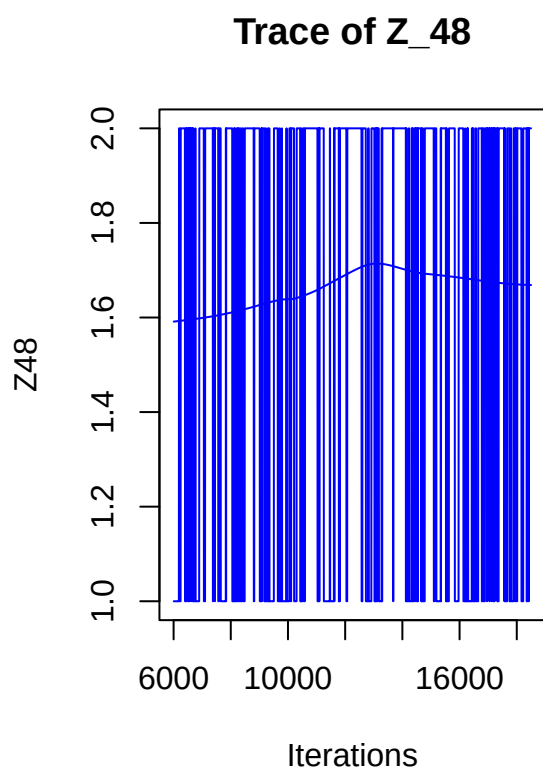
Plot the traceplots and posterior densities for latent label components Z_1 and Z_{48} , and the corresponding cluster probabilities, $p_{1,1}$ and $p_{48,2}$.

```
par(mfrow = c(2,3)) # 2 rows, 3 columns

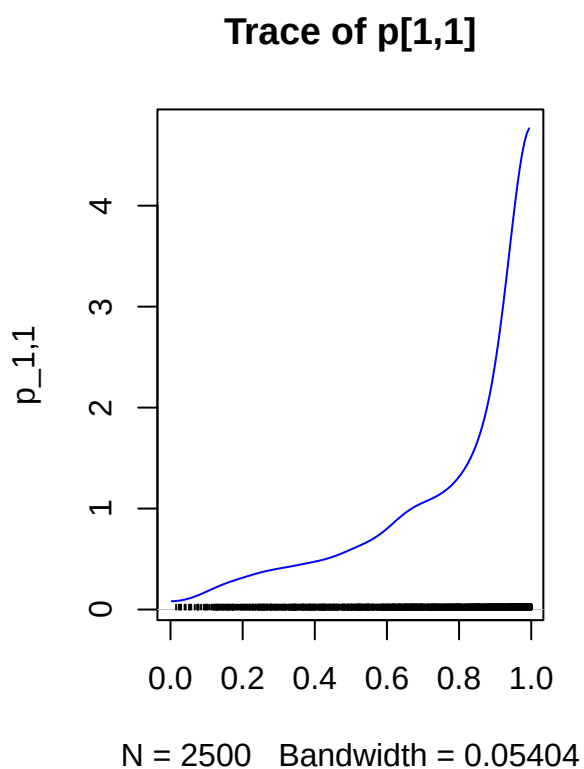
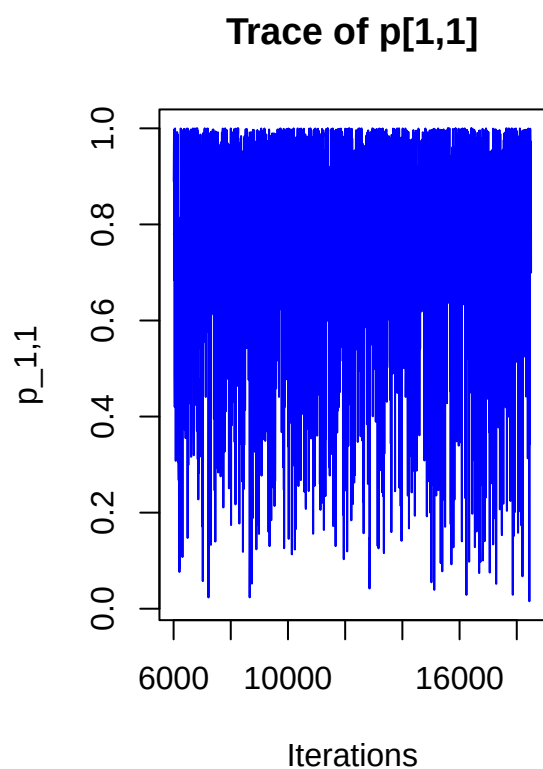
# Trace plots for sigma1, sigma2, and rho
plot(res2k[, "Z[1]"], main = "Trace of Z_1",
     ylab = "Z1", col = "blue")
```



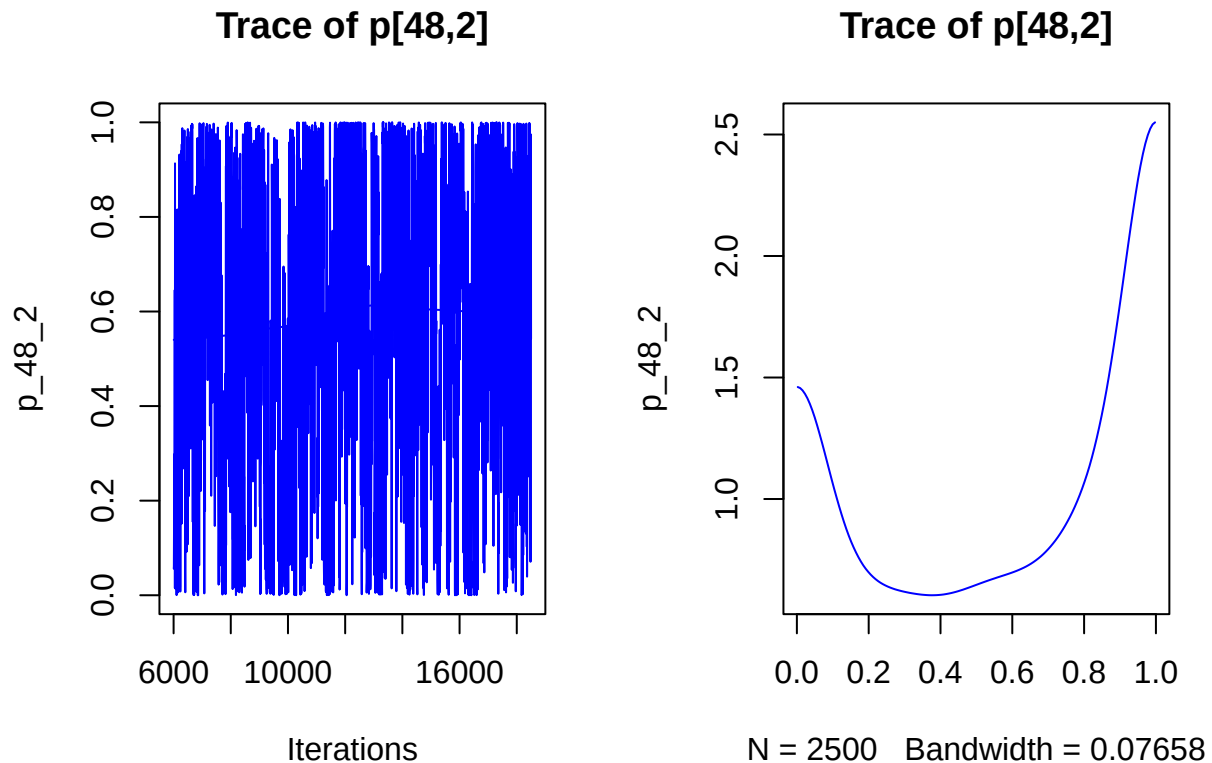
```
plot(res2k[, "Z[48]"], main = "Trace of Z_48",  
     ylab = "Z48", col = "blue")
```

```
plot(res2k[, "p[1,1]"], main = "Trace of p[1,1]",  
     ylab = "p_1,1", col = "blue")
```



```
plot(res2k[, "p[48,2]"], main = "Trace of p[48,2]",
     ylab = "p_48_2", col = "blue")
```



```
# Reset layout
par(mfrow = c(1,1))
```

The trace plots shows what has already been previously stated about the summary of the model with $K=2$:

The trace plot for Z_1 shows that observation Y_1 is always assigned to cluster 1, this is confirmed by the posterior histogram as well. Showing confidence of the classification of this observation to one of the clusters. In fact, looking at the posterior distribution of $p_{(1,1)}$, it's obvious how this is skewed towards 1 (meaning high probability of Y_1 to be assigned to cluster 1).

On the other hand, the trace plot for Z_{48} shows uncertainty in the assignment of the cluster, showing frequent switching between cluster 1 and 2. This is stressed by the posterior histogram as well: most of the samples for Y_{48} are assigned to cluster 2, but a good proportion is classified as cluster 1. Also, looking at the posterior of $p_{(48,2)}$ it can be noticed how it is more spread out between 0 and 1, with a dip in the middle (meaning that the model assigns significant probability to both cluster for Y_{48}). This suggest that on some observations the model is uncertain on which cluster to choose.

Adapt JAGS code for a general $K = K$ mixture model.

Include the MFCCs_.2 and MFCCs_11 coefficients for frog 56. Fit three mixture models to these data, with number of mixture components $K = 2$, $K = 3$, and $K = 4$.

```
model_string_k <- "model{
  #firs element of mu, sampled k times and then sorted
```

```

for(i in 1:k){
  mu1_start[i] ~ dnorm( mu00, tau00 )
  mu2[i] ~ dnorm( mu00, tau00 )
}

mu1[1:k] <- sort(mu1_start)

#precision for sigma squared 1 and 2
tau1 ~ dgamma(a,b)
tau2 ~ dgamma(a,b)

#rho
rho ~ dunif(c,d)

#compute precision matrix (inverse of covariance matrix)
prec[1,1] <- tau1 / (1-rho^2)
prec[2,2] <- tau2 / (1-rho^2)
prec[1,2] <- -rho * sqrt(tau1 * tau2) / (1-rho^2)
prec[2,1] <- prec[1,2]

#Matrix of mu used later (each row (1 to k) is a different mu)
for (i in 1:k) {
  mu[i,1] <- mu1[i] # First dimension mean
  mu[i,2] <- mu2[i] # Second dimension mean
}

for(i in 1:n){

  #each observation has a different probability for clusters:

  # p is a matrix with n rows, on each row we have a prob vector of k elements
  #(for the k clusters)

  p[i,1:k] ~ ddirch(rep(0.5, k))
                                #(non-informative prior)

  #cluster for observation i
  Z[i] ~ dcat(p[i,1:k])

  Y[i,1:2] ~ dmnorm( mu[Z[i],1:2], prec )
}

sigma_1 <- sqrt(1/tau1)
sigma_2 <- sqrt(1/tau2)
}"

```

In the previous model, the number of clusters was fixed at $K=2$, $p_{(i,1)}$ was assigned a Beta prior and $p_{(i,2)}$ was computed constraining the two probabilities to sum to 1. The model “model_string_k” introduce a general approach for a number K of desired clusters. Loops are used to generate the structure.

The probabilities for each observation to belong to each cluster, $p_{(i,k)}$, are now assigned a Dirichlet prior, that ensures that they sum to 1. This replaces the Beta distribution used in the two-cluster case.

The Dirichlet prior is chosen to have as parameter a vector of length k , with 0.5 assigned for all the clusters

probabilities.

The model structure remains the same in terms of covariance assumptions: all clusters share a common covariance matrix. The vector of $\mu_1^{(k)}$ is sorted as before, to prevent identifiability issues.

Also, the priors for all the other parameters are the same as before. A small difference is that the means are now assigned a prior using `dnorm` and not the multivariate function `dmnorm`, but since they are independent, it makes no difference.

```
model_string_ALTERNATIVE <- "model{

  #firs element of mu, sampled k times and then sorted

  for(i in 1:k){
    mu1_start[i] ~ dnorm( mu00, tau00 )
  }

  mu1[1:k] <- sort( mu1_start ) #sorting first element of mu

  #second element of mu (sampled k times)

  for(i in 1:k){
    mu2[i] ~ dnorm( mu00, tau00 )
  }

  #precision for sigma squared 1 and 2
  tau1 ~ dgamma(a,b)
  tau2 ~ dgamma(a,b)

  #rho
  rho ~ dunif(c,d)

  #compute precision matrix (inverse of covariance matrix)
  prec[1,1] <- tau1 / (1-rho^2)
  prec[2,2] <- tau2 / (1-rho^2)
  prec[1,2] <- -rho * sqrt(tau1 * tau2) / (1-rho^2)
  prec[2,1] <- prec[1,2]

  #Matrix of mu used later (each row (1 to k) is a different mu)
  for (i in 1:k) {
    mu[i,1] <- mu1[i] # First dimension mean
    mu[i,2] <- mu2[i] # Second dimension mean
  }

  for(i in 1:n){

    #each observation has a different probability for clusters:

    #for each observation (i) sample k beta(e,f) and keep track of sum of a
    for(j in 1:k){
      g[i,j] ~ dbeta( 0.5, 0.5)
    }
  }
}
```

```

g_tot[i] <- sum(g[i,])

#normalise the a vector by its sum, to obtain for each observation(i) a
#vector of sum 1. Proportional to a, so proportion of probability between
#each cluster is kept!

# p is a matrix with n rows, on each row we have a prob vector of k elements
#(for the k clusters)

for(j in 1:k){
  p[i,j] <- g[i,j] / g_tot[i]
}

#cluster for observation i
Z[i] ~ dcat(p[i,])

Y[i,] ~ dmnorm( mu[Z[i],], prec )
}

sigma_1 <- sqrt(1/tau1)
sigma_2 <- sqrt(1/tau2)
}"

```

This is an alternative model to the generalized “model_string_k”, instead of using a Dirichlet prior, the probabilities $p_{(i,k)}$ are computed assigning a beta prior to additional parameters $g_{(i,k)}$, then for each observation i , the sum of the $g_{(i,k)}$ is computed over the K cluster, and then it’s used to normalize the $g_{(i,k)}$ in order to obtain a vector of probabilities that sum to 1 for each i . The model is NOT used in the next step, since after testing it showed lower level of performances.

```

#update Y, adding coefficients from frog 56 and scaling

index_56 <- which( frogs$RecordID == 56)

coeff_56 <- cbind(frogs[index_56,]$MFCCs_.2, frogs[index_56,]$MFCCs_11)

Y_new <- rbind( Y, coeff_56)

Y_new_scaled <- scale(Y_new)

```

The models with the different numbers of $K=2,3,4$ can now be fitted.

```

#fit model with k=2
mu00 <- 0
tau00 <- 1/(100^2)
n_new <- nrow(Y_new_scaled)

k <- 2

data22 <- list(Y = Y_new_scaled, mu00 = mu00, tau00 = tau00, a = a, b = b,
              c = c, d = d, n = n_new, k = k)

model_k22 <- jags.model(textConnection(model_string_k),
                       n.chains = 2, data = data22)

```

```
## Compiling model graph
##   Resolving undeclared variables
##   Allocating nodes
## Graph information:
##   Observed stochastic nodes: 119
##   Unobserved stochastic nodes: 245
##   Total graph size: 515
##
## Initializing model
```

```
update(model_k22, 15000, progress.bar = "none")
```

```
#fit model with k=3
```

```
k <- 3
```

```
data3 <- list(Y = Y_new_scaled, mu00 = mu00, tau00 = tau00, a = a, b = b,
             c = c, d = d, n = n_new, k = k)
```

```
model_k3 <- jags.model(textConnection(model_string_k),
                      n.chains = 2, data = data3)
```

```
## Compiling model graph
##   Resolving undeclared variables
##   Allocating nodes
## Graph information:
##   Observed stochastic nodes: 119
##   Unobserved stochastic nodes: 247
##   Total graph size: 519
##
## Initializing model
```

```
update(model_k3, 15000, progress.bar = "none")
```

```
#fit model with k=4
```

```
k <-4
```

```
data4 <- list(Y = Y_new_scaled, mu00 = mu00, tau00 = tau00, a = a, b = b,
             c = c, d = d, n = n_new, k = 4)
```

```
model_k4 <- jags.model(textConnection(model_string_k),
                      n.chains = 2, data = data4)
```

```
## Compiling model graph
##   Resolving undeclared variables
##   Allocating nodes
## Graph information:
##   Observed stochastic nodes: 119
##   Unobserved stochastic nodes: 249
##   Total graph size: 523
##
## Initializing model
```

```
update(model_k4, 15000, progress.bar = "none")
```

Evaluate the DIC for all three model fits, using 5000 samples. Which model fits the data better?

```
dic_k22 <- dic.samples(model_k22, thin=5, n.iter =5000)
dic_k3 <- dic.samples(model_k3, thin=5, n.iter =5000)
dic_k4 <- dic.samples(model_k4, thin=5, n.iter =5000)
```

```
print("K=2:")
```

```
## [1] "K=2:"
```

```
print(dic_k22)
```

```
## Mean deviance: 443.1
## penalty 627.7
## Penalized deviance: 1071
```

```
print("K=3:")
```

```
## [1] "K=3:"
```

```
print(dic_k3)
```

```
## Mean deviance: 342.8
## penalty 38.15
## Penalized deviance: 381
```

```
print("K=4:")
```

```
## [1] "K=4:"
```

```
print(dic_k4)
```

```
## Mean deviance: 131.4
## penalty 95.4
## Penalized deviance: 226.8
```

The best model is the one with lowest DIC (k=4)

```
#function to compute mode
mode_of_vector <- function(x) {
  frequency <- table(x) # Count occurrences
  mode_value <- as.numeric(names(frequency)[which.max(frequency)])
  return(mode_value)
}
```



```

samples_Z <- coda.samples(model_k4,
                          variable.names = c("Z"),
                          n.iter = 10000)

# Save sample as a matrix (each row is a sample)
Z_samples <- as.matrix(samples_Z)

print(dim(Z_samples)) #MODEL IS TWO CHAINS!

## [1] 20000 119

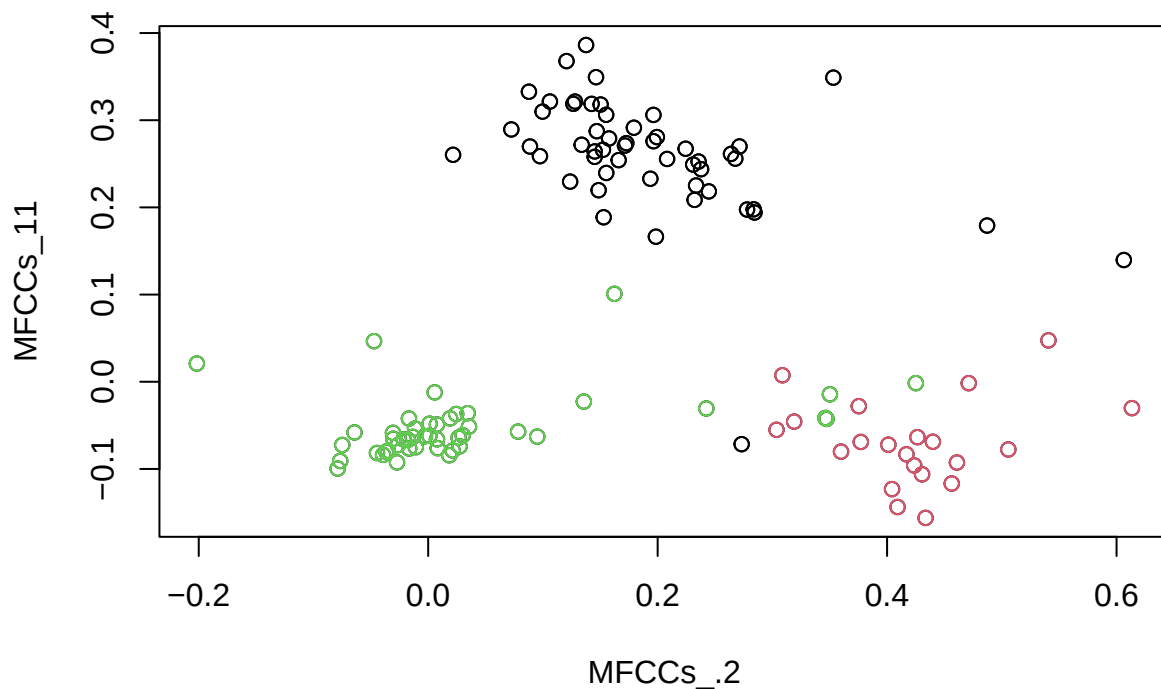
# Compute posterior mode for each syllable (most frequent value)
posterior_mode_Z <- apply(Z_samples, 2, function(x) mode_of_vector(x))

```

The best model from the previous step ($K=3$ clusters) is used to sample the latent variable Z_i , representing the cluster assignment, for each observation relative to frog 1,46 and 56 in each one of the samples.

All the samples are stored in a matrix ($n.iter=10000$, $n.chain=2$, and number of observation Y_i is 119, meaning that the sample matrix will have 5000 rows and 119 columns). In order to compute the mode for each observation then the apply function is used on the columns, applying the function “mode_of_vector” that determines the mode of a given vector.

Below is a plot with each point coloured according to the true frog label (black = 1, red = 46, green = 56).



Now we do a plot based on our clusters result.

```

mode_1_index <- which(posterior_mode_Z == 1) # Index of observations
mode_2_index <- which(posterior_mode_Z == 2) #cluster 2

mode_3_index <- which(posterior_mode_Z == 3) #cluster 3

mode_4_index <- which(posterior_mode_Z == 4) #cluster 4

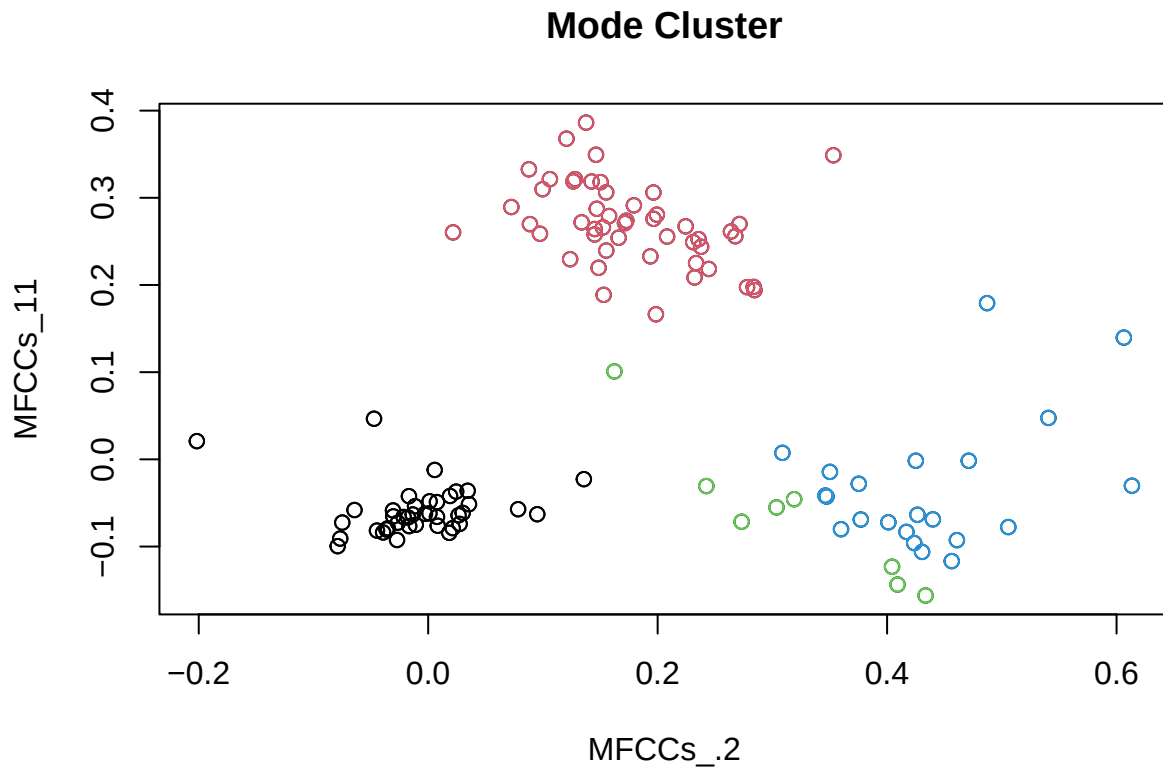
# color vector for plotting
col_tot <- rep(0, N) #set all to 0 initially

#assign colors based on cluster assignment
col_tot[mode_1_index] <- rep(1, length(mode_1_index))
col_tot[mode_2_index] <- rep(2, length(mode_2_index))
col_tot[mode_3_index] <- rep(3, length(mode_3_index))
col_tot[mode_4_index] <- rep(4, length(mode_4_index))

# Plot data points
plot(subset.data, main = "Mode Cluster")

# Add colored points based on cluster assignment
points(subset.data, col = col_tot, asp = 1)

```



The plot represents the clustering results obtained from the mixture model with $K=4$.

The x-axis corresponds to the MFCCs_2 coefficient, while the y-axis represents MFCCs_11. Each point in

the scatter plot represents an observation, and the colors indicate the assigned clusters based on the posterior mode of the latent variable Z , that was computed earlier. Clustering is not the same as the one given by dividing by frogs.